Report for "Numerical Simulations in Financial Mathematics Applications: Asset Pricing Modeling with Deep Learning"

H. Noah Belzer

Bedřich Sousedík, PhD

May 24, 2025

Department of Mathematics and Statistics

University of Maryland, Baltimore County (UMBC)

Abstract

In this study, we used deep neural networks, including Feedforward Neural Networks (FFN), Recurrent Neural Networks (RNN) with Long Short-Term Memory (LSTM), and Generative Adversarial Networks (GAN), to create an estimated asset pricing model for stock returns. Then, considering the no-arbitrage condition, we utilized sample test assets and determined economic states using macroeconomic time series datasets to produce a new asset pricing model with improvements to the Sharpe ratio, pricing errors, and explained variation. Additionally, we used numerical simulations, statistics, and machine learning techniques to forecast future trends among the macroeconomic indicators. Using the new asset pricing model and forecasts, we created a dynamic portfolio optimization program that updates with the latest data. Our research may inspire broader usage of mathematical techniques for investing.

Table of Contents

1 Acknowledgments and Funding **2**

2 Introduction **3**

3 Background **4**
3.1 Economic Theory 4
3.2 Deep Learning Concepts 5
3.3 Essential Question 5

4 Methodology **6**

5 Model Metrics and Results **7**

6 Conclusion **8**

References **9**

1 Acknowledgments and Funding

This work was funded through an Undergraduate Research Award from the UMBC Office of Undergraduate Education. We received a total of $1000 to complete the research, and we used the full amount in the following way:

- Textbooks: $98 ($11 + $22 + $65)
- Poster from CommonVision: $28.70
- Samsung External Storage Drive: $90
- GitHub Copilot Pro: $20 ($10 + $10 for a 2-month subscription)
- Microsoft 365 Family Storage (for 1 TB of storage): $130
- Student Living Stipend (food, gas, etc.): $633.30

2 Introduction

Statement of Purpose

The purpose of this research project is to study financial mathematics concepts at the undergraduate level, which will include concepts from several areas of mathematics, statistics, and computer programming. The central focus will be on portfolio optimization, which involves selecting the best portfolio from a group of candidates to achieve specific objectives, such as maximizing returns while minimizing risks and losses. This process requires addressing multiple tasks simultaneously, making it a multi-objective optimization problem. As a result, we can use mathematical concepts to optimize a portfolio that must meet those objectives. The research will draw on specific concepts from Differential Equations, Numerical Analysis, Probability and Statistics, Numerical Simulations, and Data Analysis, that are not typically covered in-depth in undergraduate mathematics and statistics courses at UMBC, particularly in the context of financial applications and niche areas like portfolio optimization. Our goal is to find historical investment/portfolio data to perform data analysis and numerical simulations to address the multi-objective optimization problem. We will use a combination of math and computer programming to find a way to optimize portfolios. By completing this research, we hope to gain both theoretical and practical experience in financial mathematics.

Motivation

Financial mathematics is very interesting as it draws on a wide range of mathematical, statistical, and programming concepts, along with insights from economics, business, and psychology. Any person or business with investments and savings can benefit from financial math concepts. Additionally, numerical simulations are very important in portfolio optimization, particularly in estimating expected values of random variables at specific times, known as exercise dates. This estimation involves dealing with stochastic equations, where the noise term is driven by geometric Brownian motion. In general, portfolio optimization will be very beneficial for individual investors who want to maximize earnings from their stocks, it can help people conserve their savings and minimize investing risks, and financial companies already use these concepts to provide their clients with adequate service. Our project will be specific because it will use numerical simulations and a data set containing historical data, but the scope of the project can be very large and significant, especially since large financial companies make use of mathematical models. The knowledge we gain will be broad and have many applications.

3 Background

3.1 Economic Theory

First, we will provide an overview of the theoretical and overarching economic concepts that form the foundation of this project. We will include explanations and any necessary mathematics for each concept. This background information will help provide a basic understanding of the concepts that guided our project's methodology.

Arbitrage: A strategy in which a trader simultaneously buys and sells the same asset in different markets, taking advantage of its different prices to ensure a risk-free profit.

No-Arbitrage Pricing Theory: In an efficient market, there is no opportunity for riskless profit. If an asset were mispriced (creating an arbitrage opportunity), traders would immediately exploit it, pushing the price back to equilibrium.

Stochastic Discount Factor (SDF): A function that accounts for the stochastic (random) nature of economic outcomes to "discount" future asset payoffs based on both time and risk, in order to determine the asset's present value and fair price.

$$M_{t+1} = 1 - \omega_t^\top R_{t+1}^e$$

M_{t+1} = SDF
N = number of assets
ω_t = vector of asset weights in portfolio
$R_{t+1,i}^e = R_{t+1,i} - R_{t+1}^f$ = excess returns
R_{t+1}^f = risk-free rate
$\omega_t^\top R_{t+1}^e$ = dot product of vectors

No-Arbitrage Assumption: The existence of an SDF, $M_{t+1} > 0$, such that, for any return in excess of the risk-free rate, it holds

$$\mathbb{E}_t[M_{t+1} R_{t+1,i}^e] = 0$$

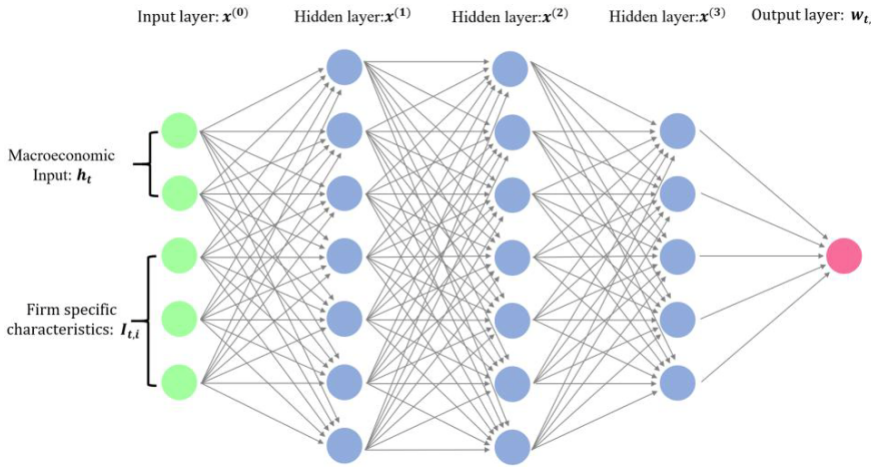
3.2 Deep Learning Concepts

Loss Function: Minimizes pricing errors and enforces the no-arbitrage condition. An adversarial network chooses $\hat{g}(I_t, I_{t,i})$ to emphasize assets with the largest mispricings. The model optimizes by minimizing the worst pricing errors. (Smaller residuals $\implies$ Better fit)

$$L(\omega \mid \hat{g}, I_t, I_{t,i}) = \frac{1}{N} \sum_{i=1}^{N} \frac{T_i}{T} \left\| \frac{1}{T_i} \sum_{t \in T_i} M_{t+1} R^e_{t+1,i} \hat{g}(I_t, I_{t,i}) \right\|^2$$

L	= loss function	I_t	= macro feature at time t
T	= total number of time periods	$I_{t,i}$	= asset-specific feature for asset i
T_i	= total number of time periods for asset i	$\sum_{i=1}^{N}$	= sum over all assets
$\hat{g}(I_t, I_{t,i})$	= selects difficult-to-price assets	$\sum_{t \in T_i}$	= sum over all time periods for asset i

Neural Network: Estimates the SDF by learning optimal portfolio weights. This image shows three hidden layers, but this can vary. The optimal model uses two.



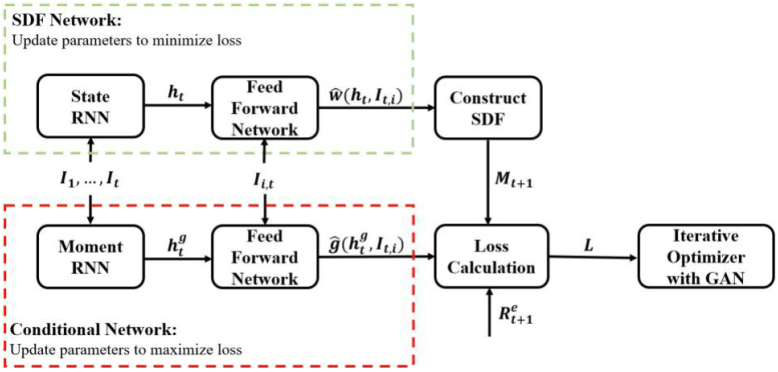
3.3 Essential Question

One of the main questions we addressed in this research was "Why do different assets have different expected returns?" The answer is, according to No-Arbitrage Pricing Theory, expected returns differ because assets have different exposure to the SDF. We estimate the SDF that explains all stock returns from the conditional moment constraints implied by no arbitrage.

4 Methodology

We use a general nonlinear asset pricing model created by Chen, Pelger, and Zhu, with deep neural networks. We utilize monthly U.S. stock data from 1967 to 2016, 178 macro-economic time series and 46 firm-specific characteristics as the input datasets.

Generative Adversarial Network (GAN) Model Architecture: An adversarial network is a model that uses two competing networks that optimize in opposing directions. Here, it is the SDF Network and the Conditional Network. The conditional network selects asset weights that maximize pricing errors to identify the most mispriced assets, forcing the SDF Network to minimize the worst-case errors.



SDF Network: Uses a State Recurrent Neural Network (SRNN), which processes past feature information $I_1, ..., I_t$ to generate a hidden state h_t. A Feed Forward Network (FFN) then uses h_t along with asset-specific information $I_{t,i}$ to produce SDF weights $\hat{w}(h_t, I_{t,i})$. The constructed SDF is then applied to asset returns.

Conditional Network: Learns an adversarial weighting function that maximizes pricing errors, identifying worst-case mispricings. The Moment RNN processes past feature information $I_1, ..., I_t$ to produce a hidden state h_t^g. An FFN takes h_t^g and asset-specific information $I_{t,i}$ to compute weights $\hat{g}(h_t^g, I_{t,i})$.

Loss Calculation and Iterative Optimizer: The loss function L quantifies pricing errors. The SDF Network minimizes L for accurate pricing, while the Conditional Network maximizes L to identify worst-case mispricings.

5 Model Metrics and Results

(Unconditional) Sharpe Ratio (SR): Measures the risk-adjusted return of the SDF portfolio by comparing its expected return to its volatility (standard deviation).

$$\text{SR} = \frac{\mathbb{E}[\text{portfolio return}]}{\sqrt{\text{Var}[\text{portfolio return}]}} \qquad \text{(Higher value } \implies \text{ More efficient model)}$$

Explained Variation: Measures the proportion of the total variation in excess returns that is explained by the SDF. (Higher value $\implies$ Better performance)

$$EV = 1 - \frac{\left(\frac{1}{T}\sum_{t=1}^{T}\frac{1}{N_t}\sum_{i=1}^{N_t}(\epsilon_{t+1,i})^2\right)}{\left(\frac{1}{T}\sum_{t=1}^{T}\frac{1}{N_t}\sum_{i=1}^{N_t}(R^e_{t+1,i})^2\right)}$$

ϵ = pricing error for asset i at time $t+1$

(Weighted) Cross-Sectional Mean: Measures how well the model explains the cross-sectional variation in asset returns across different time periods. (Higher value $\implies$ Stronger explanatory power)

$$XS - R^2 = 1 - \frac{\frac{1}{N}\sum_{i=1}^{N}\frac{T_i}{T}\left(\frac{1}{T_i}\sum_{t\in T_i}\epsilon_{t+1,i}\right)^2}{\frac{1}{N}\sum_{i=1}^{N}\frac{T_i}{T}\left(\frac{1}{T_i}\sum_{t\in T_i}R_{t+1,i}\right)^2}$$

Training Checkpoints and Constructing Decile Portfolios: We ran a Python training script 18 times, generating 18 checkpoint trials.

Train SDF Model: Train a GAN-based model to estimate the SDF. Compute SDF-implied weights, normalize SDF factors, and save them for portfolio construction later.

Construct Decile Portfolios: Sort assets into deciles based on predicted returns, calculate portfolio returns for each decile, and plot the performance of the decile portfolios.

Selecting the Best Checkpoints: We selected two checkpoints with the highest test Sharpe ratios. Each checkpoint includes a train, validation, and test Sharpe ratio.

	Train	**Validation**	**Test**
Checkpoint 1	4.136	1.272	0.840
Checkpoint 2	5.174	1.104	0.760

Final Metrics for the GAN Model:

	Train	Validation	Test
SDF Portfolio Sharpe Ratio	3.10	1.29	0.92
Explained Variation	0.15	0.06	0.06
(Weighted) $XS - R^2$	0.11	0.00	0.19



The plot shows the cumulative excess returns of 10 decile portfolios, calculated relative to the risk-free rate. Portfolio returns for each decile are calculated as the weighted average return of the assets in that decile.

6 Conclusion

Deep learning appears to be a promising tool that can provide insight into the underlying uncertainties associated with the portfolio optimization process. It can efficiently process large datasets to identify patterns and assist with producing portfolios with higher excess returns as compared to the risk-free rate.

In terms of successes, we were successfully able to run the code and save and analyze the corresponding visuals and model metrics. We were also able to demonstrate the strengths of using deep learning and the potential it has for the future of this field. There were no downfalls, but we could have improved the research by using different versions of the code and using a stronger computer to shorten the run times.

The research was worthwhile because we gained a significant amount of knowledge about how these different economic, mathematical, and programming concepts are intertwined. We accomplished our original goal, which was to create optimal portfolios and explore how deep learning can be utilized as a powerful tool. Overall, we hope that this research can inspire the broader usage of deep learning methodologies in other financial applications as well.

References

[1] Bodie, Z., Kane, A., & Marcus, A. (2012). *Essentials of investments*. McGraw-Hill Education.

[2] Buchanan, J. R. (2022). *An undergraduate introduction to financial mathematics* (4th ed.). World Scientific.

[3] Chen, L., Pelger, M., & Zhu, J. (2024). Deep learning in asset pricing. *Management Science, 70*(2), 714-750.

[4] Saari, D. G. (2019). *Mathematics of finance: An intuitive introduction*. Undergraduate Texts in Mathematics. Springer Nature.

[5] Sauer, T. (2017). *Numerical analysis* (3rd ed.). Pearson.

[6] Schellhorn, H., & Kong, T. (2024). *Machine learning for asset management and pricing*. Society for Industrial and Applied Mathematics.